

Contents

1	Implementation Plan: an integrated, shareable deck-build workflow	1
1.1	Summary — read this first	1
1.2	How to read this plan	3
1.3	Glossary	3
1.4	Technical Overview	4
1.5	The data flow, end to end	4
1.6	Constraints	6
1.7	Phase 0: Scaffold, descriptor, and the stage contract	6
1.8	Phase 1: The shared layers — geometry and ASAGI I/O	11
1.9	Phase 2: The stress stage	14
1.10	Phase 3: The friction stage	16
1.11	Phase 4: The material stage	18
1.12	Phase 5: The mesh stage — ingest and gate in code, build and improve via the skill	21
1.13	Phase 6: Deck assembly and the pre-flight battery	25
1.14	Phase 7: The integrated notebook and the demo project	28
1.15	Phase 8: The SAFS reproduction exercise — rebuild a production deck from raw	31
1.16	Phase 9: Second-system validation and legacy retirement	34
1.17	Testing Strategy	36
1.18	Risk Assessment	37
1.19	Suggested order and rough shape	39

1 Implementation Plan: an integrated, shareable deck-build workflow

Target folder: miniapps/seas/safs/seisol_quakeworx/integrated_workflow/ **Companion:** EXPLORE_deck_components_2026-08-01.md (read that for what exists today) **Written:** 2026-08-01

1.1 Summary – read this first

The problem. Building the inputs for a 3-D dynamic rupture simulation takes four independent ingredients: a fault mesh, a rock-property model, an initial stress field, and a friction law. Today each one is produced by a different folder of scripts, and every one of those scripts has this particular California fault baked into it — its map projection, its strike direction, its two named restraining bends, the coordinates of its hypocentre. There is no single document or notebook that shows the four together. A colleague who wants to model a different fault cannot start from what we have; they would have to read six modules and find fourteen hardcoded numbers before changing anything.

The fix. Move every fault-specific number out of the code and into one plain-text project file that describes the fault system. Give all four ingredients the same three-verb interface — build it, check it, draw it — which two of them already have. Then write one notebook that walks the four ingredients in order, with the checks shown inline as they pass or fail.

You bring raw data; the notebook builds everything else. Point it at your velocity model, your temperature model and your stress-orientation data in their raw exported form, and it constructs every gridded input file the simulator needs — then uses those freshly built files to work out the physical parameters, and writes a folder you can run. That last part matters: the tuning steps read the files that were just written, on the real mesh, so what you tune against is exactly what the simulator will read.

Expected outcome. Retargeting the workflow to a new fault system becomes editing one configuration file instead of editing code in six modules. Concretely: the fourteen hardcoded constants identified in the exploration collapse to one file; the two ingredients that already have check batteries keep them unchanged, and the two that do not get equivalents. Roughly 80% of the existing code is already fault-agnostic physics and moves across unmodified — this is mostly a hoisting and packaging job, not a rewrite. The deliverable a colleague receives is one folder they can open and run top to bottom.

The mesh is handled differently, on purpose. Three of the four ingredients are a single well-defined transformation from raw data to a file, so they belong in code. Meshing is not: there are eighteen forked mesh folders in this tree, each a one-off for a specific campaign, and no configuration file turns them into one button. But the craft is already written down, in a skill document that encodes the whole pipeline, the acceptance gates, and the failures not to repeat. So the workflow ships **two** things: a notebook that builds the three gridded ingredients, and that **skill document, which a colleague hands to Claude to build and improve their mesh.** The code owns format conversion, the gates, and the check that a finished mesh is compatible with the deck; the skill owns the judgement.

Main tradeoff / biggest risk. Meshing stays interactive. Nobody will press one button and get a production mesh for a new fault system — they will work through it with Claude, over hours, using the skill. That is an honest reflection of the problem, not a gap, but it does mean the “shareable workflow” is a notebook *plus* a guided craft, and the mesh half needs a capable agent on the other end.

How we will know it works. There is one exercise that settles it: take the raw data we already have for the San Andreas system, run it through the new workflow, and rebuild a deck we are running on Frontera today — then compare every field, value for value, against the one we built by hand. If that reproduces, the workflow is trustworthy and we have a complete worked example to hand to a colleague. If it does not, we find out exactly where, before anyone depends on it. This is Phase 8, and it is the point of the whole plan.

What this does NOT do. It targets **SeisSol only**. Every output is what SeisSol’s easi and PUMML readers expect; the MFEM spatial driver in this tree consumes the same physics via `data_projection_v1` HDF5 sidecars and is deliberately out of scope. It does not change any physics or improve any result — every equation, threshold and default stays as it is, and the acceptance test for the whole effort is that the new code reproduces an already-shipped deck **byte-for-byte**. It does not run the simulator, submit jobs, or post-process output. And it does not make the workflow correct for a fault in a different tectonic regime; it makes the *assumption* visible and configurable, but a thrust fault still needs a physics decision a human must make.

1.2 How to read this plan

- The **Summary** above is the whole idea. If you only read one section, read that.
- Each **Phase** opens with a one-sentence goal — skim those to see the shape of the work.
- The **Detailed Requirements** under each phase are the contract for the implementation agent. Read those only when you need specifics.
- The **Glossary** defines every shorthand used below.

1.3 Glossary

Label	Plain-language meaning
deck	One folder holding everything SeisSol needs for one simulation: mesh, gridded input files, YAML wiring, parameter file, receiver lists.
ASAGI nc	The NetCDF file format SeisSol reads gridded inputs from. Equidistant $x/y/z$ axes, one compound variable $data(z, y, x)$.
project descriptor stage	The new configuration file this plan introduces. One YAML file per fault system, holding every number that is currently hardcoded.
gate	One of the four ingredients (mesh, material, stress, friction) plus the fifth assembly step. Each stage exposes build / verify / plot.
the three-verb contract	One named pass/fail check inside a stage's verify battery. Existing examples: V1, F3, G2.
the mesh skill	build() writes an artifact, verify() re-opens that artifact and runs the gates, plot() draws it. Already used by the stress and friction design modules.
Stage F	code-mesh-build-improve — a document that teaches an agent to build and improve a fault-embedded tet mesh: the six-stage pipeline, the gates, and the verified failures. Shipped with the workflow; not a Python module.
data-exact reproduction	The last stage of the mesh skill: check a finished mesh against the <i>consuming deck's actual inputs</i> . The seam between meshing and this workflow, and the one mesh step that lives in code.
the reproduction exercise	The acceptance test for the whole plan: the new code, given the old inputs, writes a NetCDF whose every stored value and axis equals a shipped deck's. Stronger than a tolerance; weaker than comparing raw file bytes, which depend on the NetCDF library version. Phase 1 requirement 4 defines it.
rung (E0–E6)	Phase 8. Rebuild a production SAFS deck from its raw CVM / CTM / CSM inputs and compare field by field against the deck we run today. The plan's proof, and the workflow's worked example.
s	One step of the reproduction exercise's ladder, from ingesting the mesh (E0) to diffing a fully assembled deck (E6).
demo project	Along-strike distance in km, measured from a chosen origin along a chosen azimuth. The coordinate every along-strike design is expressed in.
	A tiny synthetic fault system that ships with the workflow so a new user can run everything without downloading the 562 MB of SAFS inputs.

Label	Plain-language meaning
legacy tree	The existing code being packaged: v3_under_construction/toolbox/combined_workflow/ and stress_build_workflow/.

1.4 Technical Overview

The deliverable is **one folder, integrated_workflow/**, holding **three things**: an importable package deckbuild/, one integrated notebook, and the vendored code-mesh-build-improve skill. deck-build vendors the verified physics from the legacy tree unchanged and adds two thin layers on top: config.py, which loads and validates a project descriptor, and contract.py, which defines the Stage interface and the GateReport object that every verify() returns.

Target: SeisSol. Outputs are ASAGI NetCDF, easi YAML, PUML/HDF5 and parameters.par. The MFEM spatial driver reads the same physics through data_projection_v1 HDF5 sidecars rather than NetCDF, so supporting it would mean a second writer behind the same stages — a clean extension, but not part of this plan.

1.5 The data flow, end to end

You bring raw data and a mesh. The notebook builds every NetCDF from that raw data — it does not ship pre-baked grids. This is a change from how the legacy toolbox works today, where combined_workflow/data/ carries an already-built safs_material_cvm.nc and safs_thermal_T.nc and the raw slices are never re-read. Rebuilding from raw is the whole point of a shareable workflow: a colleague's fault system has different raw data.

RAW (you provide)	BUILT BY THE NOTEBOOK	CONSUMED BY
-----	-----	-----
CVM velocity slices (lon,lat,vp,vs,rho)	-> material nc {rho, mu, lambda} +> plasticity nc {plastCo, bulkFriction} +> Sv(z) profile +> Qp/Qs relation	-> deck; Vs gate; Mw; L_b -> deck (if plasticity on) -> STRESS closure -> deck yaml (if Q on)
CTM temperature slices (lon,lat,T)	-> thermal nc {T} (or a depth profile, if no thermal model)	-> FRICTION zoning
CSM orientation (SHmax azimuth, R)	-> SHmax(x,y), R(x,y) (or a constant, if no stress model)	-> STRESS orientation
fault mesh, built with the skill	-> .puml.h5 + gates	-> deck; all projections

|

```

      v
STRESS nc {s_xx..s_xz}  <- orientation x Sv x closure k
FRICTION nc {rs_a, rs_srW} <- thermal nc or depth profile
      |
      v
PARAMETER DETERMINATION (reads the BAKED ncs, on the real mesh)
  kappa corridor / gate crossing -> k, f_w
  nucleation: S_E, L_b, L_nuc    -> overstress amp, radius, T_nuc
  yield margin tau/taulim        -> phi, cohesion
  Mw estimate                    -> the design as a whole
      |
      v
DECK FOLDER (ncs + yamls + par + receivers + mesh)

```

The important structural point: **parameter determination happens after baking, never before.** Every design read-out samples the written NetCDF at the real fault facets, so what you tune against is exactly what SeisSol will read. That is why the stress and friction stages must be able to rebuild their inputs cheaply — you will go round this loop many times.

Where everything lands

Every artifact has one declared home, and nothing is written anywhere else.

```

integrated_workflow/
  data/<project>/raw/          YOUR raw inputs (CVM / CTM / CSM slices, mesh)
  data/<project>/cache/        derived-but-expensive intermediates (Sv profile npz,
                              reprojected slice stacks) -- safe to delete, rebuilt
  outputs/<project>/<run_tag>/ EVERY built artifact for one parameter set
    material/ safs_material_cvm.nc, safs_plasticity_*.nc
    thermal/  safs_thermal_T.nc
    stress/   safs_stress_*.nc
    friction/ safs_friction_*.nc, safs_fault_rs_muw_*.yaml
    figures/  every plot, named after the gate or design it shows
    deck_manifest.json
  decks/<deck_name>/          the ASSEMBLED deck -- copies, not symlinks

```

run_tag is derived from the build parameters, so two parameter sets cannot overwrite each other. This is a direct response to a real failure: a shared outputs/graded_fw/ folder was reused across two notebooks and a stale, pre-edit rs_muw map was copied into a deck.

The physics modules that already exist — graded_k.py, graded_fw.py, build_friction_nc_thermal.py, build_plasticity_roten2014.py, generate_velocity_nc_from_raw.py, the seven numbered stress steps, msh_to_puml.py — move across with their module-level SAFS constants replaced by lookups on a passed-in config object. Their function bodies do not change. The notebook is deliberately thin: a parameters cell per stage, then build / verify / plot.

1.6 Constraints

Interfaces that cannot change

- The ASAGI NetCDF layout: COARDS NETCDF4, x/y/z float64, strictly increasing and **equidistant**; one compound variable data(z, y, x) with float32 members. SeisSol's easi reader defines this.
- The PUMML/HDF5 layout: geometry/connect/boundary/group datasets, the boundary = sum_i code_i << (8*i) byte packing, the face map f0={0,2,1} f1={0,1,3} f2={1,2,3} f3={0,3,2}, and positive signed tet volume.
- Gmsh output must be **v2.2 ASCII** (-format msh22). MFEM's reader has no v4 branch.
- Every existing gate name (V1–V4, F0–F6, FL, G1–G4, mesh A–E) is preserved verbatim, so existing run notes remain readable.

Dependencies

- numpy, scipy, netCDF4, h5py, pyproj, matplotlib, meshio, PyYAML. Add nothing heavier.
- gmsh is needed only by the reference mesh generator and must be an optional import.
- Everything must import and run under /usr/bin/python3 and under conda activate pythonenv, and on Colab after a pip cell.

Conventions to follow (inherited from the legacy tree — see the exploration doc §8)

- No absolute paths anywhere; the folder auto-locates itself.
- Fail loud. A missing input raises; there are no silent fallbacks or defaults-on-error.
- build() writes to outputs/, never into a shipped deck folder unless told to.
- Sign conventions: compression-**negative** Pa inside the nc, compression-**positive** MPa on the fault. z is elevation, positive up.
- verify() re-opens the written file. It never checks in-memory arrays.

Numerical constraint

- Byte-exact reproduction. Any change that perturbs a float — reordering an accumulation, switching an interpolation call, changing a float32 cast point — is a defect, not a refactor.

1.7 Phase 0: Scaffold, descriptor, and the stage contract

In one sentence: After this phase there is an installable, tested package skeleton and a file format for describing a fault system — but no physics has moved yet.

Goal

Create integrated_workflow/ with the package layout, the project-descriptor schema and loader, and the Stage / GateReport contract. Nothing here computes physics, so it is fast to review and impossible to break a result with.

Files to Create

integrated_workflow/

README.md

environment.yml

what this folder is, how to run it, how to retarget it
pinned deps (copy from combined_workflow, add pyyaml)

deckbuild/__init__.py	version string, re-exports
deckbuild/config.py	Project descriptor: dataclasses, loader, validator
deckbuild/contract.py	Stage ABC, GateReport, Gate, check(), Manifest
deckbuild/bootstrap.py	notebook bootstrap (generalised nb_bootstrap.py)
projects/demo_planar.yaml	the shipped synthetic example (Phase 7 fills it in)
projects/safs_alt.yaml	SAFS ALT, transcribed from the legacy constants
projects/safs_preferred.yaml	SAFS PREFERRED
tests/test_config.py	
tests/test_contract.py	
outputs/.gitkeep	

Detailed Requirements

1. **config.py — the descriptor.** Define frozen dataclasses and load them from YAML with `Project.load(path) -> Project`. The schema, with every field traced to the constant it replaces (see exploration doc §10):

```
@dataclass(frozen=True)
class CRS:
    epsg: str # "EPSG:32611" was: hardcoded in 5 modules
    geographic_epsg: str = "EPSG:4326"

@dataclass(frozen=True)
class StrikeFrame:
    azimuth_deg: float # 314.0 was: STRIKE_HINT_AZ / STRIKE_AZ
    origin_xy: tuple[float, float] # (606971.0, 3707270.0) was: GATE_ORIGIN_XY
    harmonise_hint_deg: float | None = None # defaults to azimuth_deg

@dataclass(frozen=True)
class GridBox:
    xmin: float; xmax: float; ymin: float; ymax: float
    zmin: float; zmax: float; dx: float; dz: float

@dataclass(frozen=True)
class NamedBand:
    s_start_km: float; s_end_km: float; name: str # was: GATE_BANDS entries

@dataclass(frozen=True)
class Hypocenter:
    # Give EITHER projected coords OR lon/lat/depth -- the loader converts via
    # ↪ cfg.crs.
    # This is a REQUESTED point; it is snapped onto the real fault in Phase 1.
    x: float | None = None
    y: float | None = None
    z: float | None = None # elevation, negative down
    lon: float | None = None
    lat: float | None = None
```

```

    depth_m: float | None = None    # positive down; converted to z with the
↪ local topo
    snap_to_l_m: float = 2000.0    # hard-fail if the fault is further than this
    label: str = ""

@dataclass(frozen=True)
class MeshSpec:
    path: str                        # .pum1.h5 relative to data/
    fault_bc: int = 3                # was: msh_to_pum1 tag map
    tag_to_bc: dict[int, int] = field(
        default_factory=lambda: {101: 3, 102: 1, 103: 5, 104: 5})
    daylights: bool = False
    daylight_min_depth_m: float = 0.0

@dataclass(frozen=True)
class PhysicsDefaults:
    mu_shear_pa: float = 23.5e9     # was: MU_SHEAR
    w_energy_m: float = 10.0e3      # was: W_ENERGY
    dc_m: float = 1.2               # was: DC
    kappa_c: float = 0.9            # was: KAPPA_C
    seis_band_km: tuple = (3.0, 12.0) # was: SEIS_BAND_KM
    active_band_km: tuple = (0.3, 15.0) # was: ACTIVE_BAND_KM
    l_coast_km: float = 6.0         # was: L_COAST_KM
    gap_max_km: float = 4.0         # was: GAP_MAX_KM
    sn_floor_mpa: float = 30.0      # was: SN_FLOOR_MPA
    rs_b: float = 0.019
    rs_sl0: float = 0.10

@dataclass(frozen=True)
class Project:
    name: str
    crs: CRS
    strike: StrikeFrame
    stress_box: GridBox
    meshes: dict[str, MeshSpec]      # {"alt": ..., "preferred": ...}
    default_mesh: str
    gate_bands: list[NamedBand]
    hypocenters: dict[str, Hypocenter] # keyed by mesh name
    physics: PhysicsDefaults
    raw: RawSources                  # see req. 2
    provenance: dict[str, str]       # free-form notes

```

2. **RawSources** names the raw inputs and, critically, their *reader kind* — so a new project can point at a different data product:

```

@dataclass(frozen=True)
class RawSources:
    orientation: SourceSpec | None # kind: "csm_csv" | "constant" | "callable"
    velocity: SourceSpec | None # kind: "cvm_slices" | "layered_1d" |
    ↪ "callable"
    thermal: SourceSpec | None # kind: "ctm_slices" | "depth_profile" | None
    # NOTE: there is no `density_profile` field. The Sv(z) profile the stress
    ↪ closure
    # needs is DERIVED from the built material nc (Phase 4), not supplied. The
    ↪ legacy
    # 6 KB density_profile.npz becomes a CACHE under data/<project>/cache/,
    ↪ keyed by the
    # material nc's sha256, and is rebuilt whenever that hash changes.

@dataclass(frozen=True)
class SourceSpec:
    kind: str
    path: str | None = None
    params: dict = field(default_factory=dict)

```

kind dispatches to a reader registered in a module-level dict. This is the extension point for a fault system with different data products.

3. **Project.validate()** must hard-fail (raise ConfigError, never warn) on:

- an unknown kind in any SourceSpec;
- default_mesh not present in meshes;
- any GridBox with $dx \leq 0$, $dz \leq 0$, $x_{max} \leq x_{min}$ (etc.);
- a hypocenters key with no matching meshes key;
- gate_bands with $s_{end_km} \leq s_{start_km}$;
- a referenced path that does not exist, when require_files=True (default True).

4. **contract.py — the three-verb contract.** Codify what graded_k and graded_fw already do informally:

```

@dataclass
class Gate:
    name: str # "V1", "F3", "G2", "M-C"
    passed: bool
    detail: str # one line, printed verbatim
    severity: str = "hard" # "hard" | "warn" | "skip"

class GateReport:
    gates: list[Gate]
    def add(self, name, passed, detail, severity="hard") -> None
    @property
    def ok(self) -> bool # all hard gates passed

```

```

def print(self) -> None      # "[PASS] V1 ..." / "[FAIL] ..." / "[ -- ] ..."
def to_dict(self) -> dict    # for the manifest
def raise_if_failed(self) -> None

class Stage(ABC):
    name: str
    def build(self, cfg: Project, out_dir: Path, **params) -> Artifact: ...
    def verify(self, cfg: Project, artifact: Artifact, **params) -> GateReport:
        ↪ ...
    def plot(self, cfg: Project, artifact: Artifact, **params): ...
    def check_outputs(self, out_dir: Path, keep: list[Path]) -> dict: ...

```

Artifact is a small dataclass: path, kind, sha256, params: dict, provenance: dict.

5. **Manifest** accumulates Artifacts and writes outputs/deck_manifest.json: for each stage, the artifact path, its sha256, the parameters that produced it, the descriptor name and its sha256, and the GateReport dict. This makes a deck reproducible and diffable.
6. **bootstrap.py** generalises lib/nb_bootstrap.py:init(). Same behaviour — locate the folder by walking up for a marker, pip-install and mount Drive on Colab, chdir, put the package on sys.path, reload every already-imported deckbuild.* module, ensure outputs/ — but the marker becomes deckbuild/__init__.py and it additionally loads the project descriptor:

```

def init(project: str = "safs_alt", require_files: bool = True) -> Bootstrap
# returns .cfg (Project), .root (Path), .out (Path), .data (Path)

```

7. **Transcribe the SAFS descriptors.** projects/safs_alt.yaml and projects/safs_preferred.yaml must reproduce, exactly, the constants listed in the exploration doc §10 table. Each field carries a YAML comment naming the legacy symbol it came from.

Edge Cases to Handle

- Descriptor references a mesh file that is absent → raise with the resolved absolute path in the message, and a hint to run download_data.py.
- require_files=False (used by tests/) must skip only file-existence checks, not schema checks.
- A YAML with an unknown top-level key → raise, do not ignore. Silent key-drop is how a retargeted project silently keeps SAFS defaults.
- Running init() twice in one kernel must be idempotent and must still reload deckbuild.*.

Acceptance Criteria

- ☐ pytest tests/test_config.py tests/test_contract.py passes.
- ☐ Project.load("projects/safs_alt.yaml") returns a Project whose every field equals the corresponding legacy constant, asserted field-by-field in a test that imports the legacy _common.py and compares.
- ☐ Project.load on a YAML with an unknown key, a bad kind, or a reversed band raises ConfigError with the offending key in the message (three separate tests).

- ❑ `GateReport.print()` output format is asserted against a golden string.
- ❑ No module in `deckbuild/` contains a literal `32611`, `314.0`, `606971`, or `3707270` — enforced by `tests/test_no_hardcoded_constants.py`, which greps the package.

Dependencies

- Depends on: nothing.
- Required by: every later phase.

1.8 Phase 1: The shared layers – geometry and ASAGI I/O

In one sentence: After this phase, the coordinate maths and the NetCDF reader/writer that all four ingredients share live in one place and take the fault description as an argument instead of assuming California.

Goal

Extract the two genuinely shared, genuinely generic layers out of the legacy tree, with their SAFS assumptions lifted into Project. These are read by every later stage, so getting them byte-exact first de-risks everything downstream.

Files to Create

- `deckbuild/geometry.py` — the strike frame and fault-facet extraction.
- `deckbuild/asagi.py` — the ASAGI NetCDF format layer.
- `tests/test_geometry.py`, `tests/test_asagi.py`.

Files to Modify

- None. The legacy tree is left untouched; Phase 1 vendors *copies*. The legacy tree is retired only in Phase 9, after reproduction is proven.

Detailed Requirements

1. **geometry.py** — port these functions from `lib/_common.py`, `lib/generate_stress_nc_from_raw.py` and `lib/project_csm_stress_to_vtu.py`, changing only their signatures:

```
def strike_s_km(x, y, strike: StrikeFrame) -> np.ndarray
    # was strike_distance_km(x, y, STRIKE_AZ, GATE_ORIGIN_XY)
    # su = (sin(az), cos(az)); s = ((x-ox, y-oy) @ su) / 1000 -- NW positive.
    # Keep the EXACT expression: with az = 314 deg this gives the x coefficient
    # -0.71933980033865119 and the y coefficient 0.69465837045899725 that appear
    # verbatim in every shipped rs_muw !LuaMap. Gate FL asserts that round-trip.
def load_fault(mesh: MeshSpec, data_dir: Path) -> Fault
    # load_puml -> extract_fault -> triangle_geometry -> harmonise_normals(hint)
    # -> tandem_basis; returns Fault(cent, normals, strikes, dips,
    #   ↪ areas)
def fault_trace(fault: Fault, depth_km=None, tol_km=0.5) -> (x, y)
def resolve_tractions(sigma, strikes, dips, normals) -> (sigma_n, tau, mu)
def build_grid(box: GridBox) -> (gx, gy, gz)
```

The `s` expression must be copied character-for-character from `strike_distance_km`; a sign flip here silently mirrors every along-strike design.

1b. **snap_hypocenter** — **put the user’s requested point ON the real fault**. A user gives a hypocentre from a catalogue, in lon/lat/depth or projected coordinates. It will not lie on the fault triangulation. Every downstream number depends on it, so snapping is a named, reported, gated operation — not a silent nearest-neighbour lookup:

```
@dataclass(frozen=True)
class SnappedHypocenter:
    xyz: tuple[float, float, float] # the point ON the fault, a facet centroid
    requested_xyz: tuple[float, float, float]
    distance_m: float # |snapped - requested|
    facet_index: int
    s_km: float # along-strike position, via strike_s_km
    depth_m: float
    report: GateReport

def snap_hypocenter(hypo: Hypocenter, fault: Fault, strike: StrikeFrame,
                    crs: CRS) -> SnappedHypocenter
```

Requirements: - Accept lon/lat/depth **or** projected x/y/z; convert with pyproj using `cfg.crs`. Raise if both or neither are given. - Snap to the nearest **facet centroid**, using a `scipy.cKDTree` over `fault.cent` with `workers=-1`. Centroid, not vertex: the friction and stress fields are evaluated per facet, so a facet is the addressable unit. - **Hard-fail if `distance_m > hypo.snap_tol_m`** (default 2 km), with the requested point, the snapped point, the distance, and the nearest facet’s depth in the message. A large snap means the user gave a point for a different fault strand, or got the CRS wrong — both are user errors that must not be silently absorbed. This has happened: one SAFS nucleation was off-fault and had to be re-snapped. - Report, in the `GateReport`, the snap distance, the resulting `s_km` and depth, and **whether the snap moved the point across a gate_band boundary** — because that silently changes which friction region the nucleation sits in. - Return the along-strike `s_km` so the friction stage’s F1 region check and the `Tnuc_s` Lua block both use the *same* snapped point, never the requested one.

2. **asagi.py** — the format layer, extracted from `generate_stress_nc_from_raw.py` and `build_friction_nc_thermal.py`, which currently each own a copy:

```
def write_asagi(path, x, y, z, fields: dict[str, np.ndarray],
               attrs: dict, dtype=np.float32) -> Path
    # fields: {"s_xx": (nz,ny,nx), ...}; writes ONE compound var `data`
    # asserts axes are float64, strictly increasing, and equidistant to 1e-9
    ↪ relative
def read_asagi(path) -> (x, y, z, dict[str, np.ndarray], attrs)
def trilinear_sample(path, px, py, pz, fields=None) -> dict[str, np.ndarray]
def hull_containment(path, pts, label="") -> GateReport
    # the G1 guard, generalised: per-axis margins, hard-fail on any point outside
def roundtrip_selfcheck(path, evaluator, n=1000, seed=12345,
```

```

median_tol=1e-5, max_tol=5e-4) -> GateReport
# the G3 guard, generalised

```

3. **Equidistance check.** `write_asagi` must reject a non-equidistant axis with a message naming the axis and the worst spacing deviation. ASAGI does not check this and produces silently wrong interpolation.
4. **The comparison harness — two tiers, and the distinction matters.** Add `tests/nc_compare.py`:

```

def compare_nc(a, b) -> NcDiff
# NcDiff.data_identical : every variable's bytes equal, axes equal, attr VALUES
↪ equal
# NcDiff.file_identical : filecmp.cmp(a, b, shallow=False)
# NcDiff.first_diff : (variable, flat index, value_a, value_b) or None
# NcDiff.attr_diffs : attributes that differ (provenance strings are
↪ expected to)

```

data_identical is the acceptance criterion; file_identical is a bonus. A NetCDF file's bytes depend on the netCDF4/HDF5 library version — chunking, compression and header layout can all change without a single stored value changing. Requiring `file_identical` would make the suite fail on a library upgrade for a reason that does not affect a single simulation. Requiring `data_identical` catches every real defect: a changed value, a changed axis, a transposed field, a shifted grid.

Where earlier phases in this plan say “byte-identical”, read it as `data_identical`, with `file_identical` reported alongside and allowed to fail with a note. This is a deliberate loosening of the original wording, and the only one — every *value* must still match exactly, with no tolerance.

Interfaces

Fault is a frozen dataclass: `cent (N,3)`, `normals (N,3)`, `strikes (N,3)`, `dips (N,3)`, `areas (N,)`, `mesh_name: str`. It is the currency between geometry and every stage.

Edge Cases to Handle

- A mesh with zero BC-3 faces → raise naming the mesh and the tag map, do not return an empty Fault.
- Degenerate facets (zero area) in `tandem_basis` → keep the legacy behaviour, which counts and reports them (`_degen`), and surface the count in the returned Fault.
- Sample points outside the nc hull in `trilinear_sample` → the legacy code edge-clamps z. Keep that, but return a boolean mask of clamped points so callers can gate on it.
- `write_asagi` with a field whose shape is `(nx,ny,nz)` instead of `(nz,ny,nx)` → raise with both shapes in the message. This transposition is an easy and catastrophic mistake.
- A hypocentre given in lon/lat when `cfg.crs.epsg` is for a different UTM zone → the snap distance will be enormous; the `snap_tol_m` hard-fail catches it, and the message must show both the projected requested point and the fault bbox so the zone error is obvious.
- A hypocentre that snaps to a facet on a *different fault strand* than intended → cannot be

detected automatically. Report the snapped facet's `s_km` and depth prominently so the user can eyeball it, and plot it on the fault in the notebook.

Acceptance Criteria

- ☐ `strike_s_km` reproduces `strike_distance_km` to 0 ULP on 10,000 random points, with the SAFS frame — asserted against the legacy import.
- ☐ `load_fault(safs_alt)` returns facet counts, centroids and normals bit-identical to `legacy_common.load_fault(ALT_MESH)`.
- ☐ `write_asagi` round-trips a random field through `read_asagi` bit-exactly.
- ☐ `write_asagi` raises on: a non-equidistant axis, a float32 axis, a transposed field (three tests).
- ☐ `trilinear_sample` on the shipped `safs_stress_andersonian_k1.7.nc` matches the legacy `generate_stress_nc_from_raw.trilinear_sample` to 0 ULP at 1000 fixed-seed points.
- ☐ `snap_hypocenter` on the SAFS ALT deck's recorded hypocentre (604446.944, 3704576.3853, -10067.9819) returns a snap distance of ~ 0 m and the `s_km` that deck's `rs_muw` map assumes.
- ☐ Given the same point as lon/lat/depth, `snap_hypocenter` returns the *same* facet.
- ☐ A point 50 km off the fault raises, and the message contains the distance and the fault bbox.
- ☐ A point placed deliberately 100 m across a `gate_band` edge produces a `GateReport` whose detail says the snap crossed a named band.

Dependencies

- Depends on: Phase 0.
 - Required by: Phases 2–6.
-

1.9 Phase 2: The stress stage

In one sentence: After this phase, the initial stress field is built and checked through the new package, and the file it writes is byte-identical to the one in a shipped deck.

Goal

Port the seven numbered stress steps and `graded_k.py` behind `StressStage`. This stage goes first because it is the most mature — it already has the V1–V4 battery and the build/verify/plot triple — so it is the cleanest test of whether the contract fits.

Files to Create

- `deckbuild/stress.py` — `StressStage`, plus the ported `step1...step7` as private functions.
- `deckbuild/orientation.py` — the orientation readers, dispatched on `SourceSpec.kind`.
- `tests/test_stress.py`.

Detailed Requirements

1. **Port the steps.** Copy `lib/step1_orientation.py ... lib/step7_project.py` and the physics they call (`magnitudes_C1`, `build_tensor_andersonian`, `shmax_from_sigma`, `csm_tensors_tension`, `csm_axes_and_shape`, `interpolate_csm_field`, `sv_total_at`, `pore_pressure`, `shape_recovery_check`) into `stress.py`. **Do not touch a single expression.** Replace only: module-level constants → `cfg.physics.*`; box literals → `cfg.stress_box`; mesh literals → `cfg.meshes[...]`.
2. **Orientation dispatch.** `orientation.py` registers readers keyed by `SourceSpec.kind`:

- "csm_csv" — the existing read_csm_csv, author SHmax azimuth (column 10) and shape ratio R (column 13). Column indices become SourceSpec.params.
- "constant" — uniform azimuth_deg and R from params. This is what a new fault system with no community stress model uses, and what the demo project uses.
- "callable" — a dotted path to a user function $f(x, y) \rightarrow (az_deg, R)$.

All three return the same (cx, cy, az_deg, R) tuple.

3. StressStage.build signature:

```
def build(self, cfg, out_dir, *,
          sv_profile,          # REQUIRED: the Artifact from MaterialStage
          k=None,              # scalar closure ratio
          design=None,         # or an N-region graded-k design
          freeze_above_depth_m=500.0,
          daylight_patch=None, # None -> cfg.meshes[mesh].daylights
          daylight_min_depth_m=None,
          mesh=None,           # None -> cfg.default_mesh
          out_name=None) -> Artifact
```

Exactly one of k or design must be given; both or neither raises.

sv_profile is a **required positional-by-keyword input, not a default path**. The stress field is orientation x Sv x closure, and Sv comes from the material the user supplied. Making it an argument means a rebuilt CVM cannot silently fail to propagate into the stress, and the artifact's provenance records which material nc it was derived from.

4. **The two shallow treatments stay exactly as they are.** Port step6_write_nc._apply_daylight_patch and the columnwise freeze verbatim. Their docstrings — which record *why* they exist (the psi_ini = -inf abort and the 446 m runaway) — must come across with them.
5. **StressStage.verify** wraps graded_k.verify, preserving V1–V4 names and their detail strings, and returns a GateReport. The V1 reference builds remain scratch and are quarantined by check_outputs.
6. **Graded-k design object.** Port graded_k's N-region design (k_values, boundaries_s_km smoothstep windows) into a KDesign dataclass in stress.py. The s coordinate comes from geometry.strike_s_km(cfg.strike) — not a local copy.

Edge Cases to Handle

- A region so narrow that its plateau k is never reached at any grid column → the legacy code warns that V1 has almost no columns to test. Keep the warning, verbatim.
- freeze_above_depth_m = 0 → freeze off, V3 reports skip, not pass.
- A fault that daylights but daylight_patch=False → emit a **hard warning** naming the facet count that will get sigma_n = 0, because that is the psi_ini = -inf abort.
- $k < 1$ anywhere → raise. The closure algebra assumes $\sigma_1 \geq \sigma_3$.

Acceptance Criteria

- ☐ `StressStage.build(safs_alt, k=1.7, freeze_above_depth_m=0)` writes a file **byte-identical** to the shipped `safs_seisol_v4_0_0..._deep40km/safs_stress_andersonian_k1.7.nc`, via `assert_nc_identical`.
- ☐ The same for one graded-k deck — the CASE2 `kse1p40_knw1p25` one:
```\nsafs\_seisol\_v4\_0\_0\_RSSRW\_ALT\_THERMAL\_CASE2\_small\_plasticity\_phi30\n \_gradedfw\_kse1p40\_knw1p25\_attenuation\n```\
- ☐ `verify` reproduces the V1–V4 pass/fail vector of the legacy `graded_k.verify` on both decks.
- ☐ `build(..., kind="constant")` on the demo project produces a valid nc that passes V2 (the eigen-closure check) — proving the stage works with no California data.
- ☐ Both error paths raise: `k` and `design` both given; `k < 1`.
- ☐ Building with an `sv_profile` derived from a rebuilt-from-raw CVM gives a stress nc byte-identical to building with the one derived from the shipped CVM.
- ☐ Omitting `sv_profile` is a `TypeError`, not a silent default.

## Dependencies

- Depends on: Phases 0, 1, and **Phase 4** for `sv_profile`. Develop against the shipped CVM's profile first; the raw path closes when Phase 4 lands.
  - Required by: Phase 6.
- 

## 1.10 Phase 3: The friction stage

**In one sentence:** After this phase, the rate-and-state friction fields are built and checked through the new package — and a fault system with no community thermal model has a supported way to specify them.

### Goal

Port `build_friction_nc_thermal.py`, `friction_build.py` and `graded_fw.py` behind `FrictionStage`, and add the depth-profile source that the exploration doc flagged as missing (open question 5).

### Files to Create

- `deckbuild/friction.py` — `FrictionStage`, the  $a(T) / V_w(T)$  profiles, the zone surgery, the `graded-f_w` design and its Lua emitter.
- `tests/test_friction.py`.

### Detailed Requirements

1. **Port the thermal profiles verbatim** — `a_minus_b(T, case)`, `a_of_T`, `vw_of_T`, and the closed-form unit self-test that runs *before* any baking. The kink temperatures (50/100/150/300/350/400) and the slope  $m = 8.0e-5$  become `FrictionSpec` fields with those defaults, not module con-

stants.

2. **Add a depth\_profile friction source.** New kind for `RawSources.thermal`:

```
projects/demo_planar.yaml
thermal:
 kind: depth_profile
 params:
 a_minus_b_knots_m: [0, -3000, -12000, -15000]
 a_minus_b_values: [0.004, -0.004, -0.004, 0.004]
 vw_knots_m: [0, -14000, -16000]
 vw_values: [0.05, 0.05, 1000.0]
```

Piecewise-linear in depth, evaluated on the same grid the thermal source would have produced. This is the fallback for a fault system with no CTM. It reuses the *same* `write_friction_nc` and the *same* G1–G4 guards.

3. **FrictionStage.build** signature:

```
def build(self, cfg, out_dir, *,
 case=1, # thermal zoning case (kind="ctm_slices")
 vs_barrier=None, # BarrierSpec | None
 vw_patches=(), # tuple[PatchSpec, ...] (CASE2_SPECIAL
 ↪ surgery)
 mesh=None, out_name=None) -> Artifact
```

4. **Graded f<sub>w</sub>.** Port `graded_fw.py`'s N-region design, `write_lua_map`, and the F0–F6 + FL battery. The emitted Lua currently hardcodes the azimuth and origin inline; it must now be **templated from `cfg.strike`**, and the emitter must assert that the numbers it wrote parse back to `cfg.strike` values (this is gate FL).
5. **Keep the SeisSol version gate.** Port section [0] of the `graded-fw` notebook into `friction.check_seissol_supports_spatial_muw(src_path) -> GateReport`. Spatial `rs_muw` needs SeisSol newer than v1.3.2; v1.1.3 accepts the YAML and silently ignores it. This gate must run **before** build, and its detail must name the version it found.
6. **Zone surgery** — port `apply_vw_ceiling_patches` and `apply_vs_barrier` unchanged, with their cos-taper geometry expressed in `s/depth` via `geometry.strike_s_km`.
7. **Consume the thermal nc built by Phase 4, not a shipped one.** `FrictionStage` takes the thermal artifact as an input, and records its sha256 in its own provenance. If the user supplied CTM raw slices, that nc was built this run; if they supplied a depth profile, there is no thermal nc and the profile is evaluated directly on the friction grid. Either way the friction stage must never read a file it did not receive as an argument.
8. **The nucleation block uses the SNAPPED hypocentre.** The `Tnuc_s !LuaMap` embeds the hypocentre coordinates inline. Those must be `SnappedHypocenter.xyz` from Phase 1, never the user's requested point — otherwise the compact bell  $F(r) = \exp(r^2/(r^2-R^2))$  is centred slightly off the fault and the peak overstress lands on no facet at all. The emitter must

assert the embedded coordinates equal the snapped ones, and F6's report must print the snap distance alongside S\_E.

### Edge Cases to Handle

- $V_w$  surgery that *lowers*  $V_w$  anywhere → raise. The legacy contract is that  $V_w$  is only ever raised; lowering it silently destabilises the deep VS zone.
- A nucleation radius smaller than the local facet size → warn with both numbers: the bell will be resolved by one or two facets and the effective overstress will be mesh-dependent.
- $rs_a \leq 0$  after any surgery → hard fail (G2).
- A `depth_profile` whose knots are not monotonically decreasing in  $z$  → raise.
- A graded- $f_w$  region narrow enough that its plateau is never reached → the legacy code warns that F1/F2 will have almost nothing to test, and degrades `fw_gate`. Keep both behaviours and both messages.
- $f_w \geq f_0$  in any region → hard fail (F0).

### Acceptance Criteria

- ☐ `FrictionStage.build(safs_alt, case=1)` writes a file **byte-identical** to the shipped `safs_friction_thermal_case1.nc`.
- ☐ Same for `case=2` against a CASE2 deck.
- ☐ G1–G4 reproduce the legacy pass/fail vector, including the G3 kink-straddle exemption (a test that plants a synthetic excursion in a kink-free cell must **fail**).
- ☐ The emitted Lua for the shipped 6-transition `rs_muw` map is byte-identical to `safs_fault_rs_muw_fw0-0-0.045-0.03-0.06-0.0175-0.05_s20-28...yaml`.
- ☐ The `depth_profile` source produces a valid nc on the demo project that passes G1–G3.
- ☐ `check_seissol_supports_spatial_muw` returns `passed=False` for a v1.1.3 tree and `True` for master (tested against two pinned version strings, not a live checkout).
- ☐ The emitted `Tnuc_s` Lua embeds exactly `SnappedHypocenter.xyz`, asserted against the shipped ALT deck's (604446.944, 3704576.3853, -10067.9819).
- ☐ Building with thermal from a rebuilt-from-raw CTM nc gives a friction nc byte-identical to building from the shipped `safs_thermal_T.nc` — proving the raw rebuild is faithful.

### Dependencies

- Depends on: Phases 0, 1. The `ctm_slices` path depends on Phase 4, which builds the thermal nc; the `depth_profile` path does not.
- Required by: Phase 6.

---

## 1.11 Phase 4: The material stage

**In one sentence:** After this phase, the rock-property file, the plasticity file derived from it, and the attenuation parameters are all produced by one stage with one grid definition.

### Goal

Port three legacy modules behind `MaterialStage`, and give the material a gate battery it currently lacks. The three are:

generate_velocity_nc_from_raw.py	raw CVM slices	-> material nc
generate_thermal_nc_from_raw.py	raw CTM slices	-> thermal nc
build_plasticity_roten2014.py	material nc	-> plasticity nc

## Files to Create

- deckbuild/material.py — MaterialStage, the slice reader, the moduli conversion, the Roten-2014 derivation, the Q relation.
- tests/test\_material.py.

## Detailed Requirements

1. **Port the two-stage raw pipeline verbatim.** Slice read → pyproj reproject → inscribed UTM grid → LinearNDInterpolator per slice per field → depth-to-elevation flip → surface clone. Then moduli **at the source nodes** ( $\mu = \rho * V_s^2$ ,  $\lambda = \rho * (V_p^2 - 2 * V_s^2)$ ), then the linear z-resample. The order matters: it keeps the stored grid the exact piecewise-linear interpolant of node moduli. Add a comment saying so, because it looks like an inefficiency.

2. **Velocity source dispatch**, keyed on RawSources.velocity.kind:

- "cvm\_slices" — the existing reader; column names go in params.
- "layered\_1d" — depth\_m, vp, vs, rho knot lists; laterally uniform. The demo project uses this.
- "callable" — a dotted path to  $f(x, y, z) \rightarrow (vp, vs, \rho)$ .

2b. **Thermal source dispatch**, keyed on RawSources.thermal.kind. MaterialStage owns this because it is the same two-stage recipe on a different scalar: - "ctm\_slices" — port generate\_thermal\_nc\_from\_raw.py; produces data(z,y,x){T}. Its own defaults (dz=200, z\_min=-21000, extend\_z\_top=200) are independent of the velocity grid's — the two grids do **not** have to match, and must not be forced to. - "depth\_profile" — no nc is built; the friction stage evaluates the profile directly. - None — no thermal model; FrictionStage must then be given a depth profile or it raises.

3. **MaterialStage.build** produces up to four artifacts, controlled by flags:

```
def build(self, cfg, out_dir, *,
 grid_dx=1500.0, dz=250.0, z_min=-45000.0, z_max=100.0,
 extend_z_top=100.0, dtype=np.float32,
 plasticity: PlasticitySpec | None = None,
 attenuation: AttenuationSpec | None = None,
 thermal: bool = True) -> MaterialArtifacts
MaterialArtifacts: .material .plasticity .thermal .sv_profile (Artifact|None)
```

PlasticitySpec(phi\_soft\_deg=35, phi\_hard\_deg=45, vs\_threshold=2500.0, cohesion\_factor=1e-4) — the production decks use 30/40, the paper uses 35/45, so both must be reachable and neither may be the silent default. Default to the **paper** values and require the deck to state otherwise. AttenuationSpec(qs\_over\_vs=0.05, qp\_over\_qs=2.0, freq\_central=0.5, freq\_ratio=100.0) — this stage only *emits the YAML block and records the numbers*; the Q fields are computed by SeisSol from  $\mu/\rho$ .

- 3b. **Derive the Sv(z) profile here, and hand it to the stress stage.** The vertical effective stress

the closure needs is a property of the material, so it is built where the material is:

```
def sv_profile(material_nc, g=9.81, pore="hydrostatic") -> Artifact
lateral-mean rho(z) over the grid -> trapezoidal integration -> Sv_total(z)
-> subtract hydrostatic Pp -> Sv_eff(z). Writes a small npz + records the
material nc's sha256 in its provenance.
```

Cached under `data/<project>/cache/sv_profile_<material_sha256[:12]>.npz` and rebuilt whenever the material hash changes, so a rebuilt CVM can never be paired with a stale profile. The legacy `density_profile.npz` shipped in `combined_workflow/data/` becomes a *test fixture* proving this derivation reproduces it, not an input.

4. **New gate battery M1–M5** (the material currently has only the round-trip self-check):

- |    |                                                                          |      |                                        |
|----|--------------------------------------------------------------------------|------|----------------------------------------|
| M1 | <code>lambda &gt; 0</code> everywhere                                    | HARD | (legacy: a bare exception)             |
| M2 | no NaN cells after interpolation                                         | HARD | (legacy: "fails loud", now ↪ named)    |
| M3 | mesh containment: every mesh node inside the grid hull, per-axis margins |      |                                        |
|    |                                                                          | HARD | for the fault bbox, WARN for far field |
| M4 | fixed-seed round-trip self-check                                         | HARD | (the existing one, named)              |
| M5 | Vs range plausible: $100 < V_s < 8000$ m/s                               | WARN | with the offending percentile          |

M3 must distinguish the fault bbox from the absorbing perimeter, because the shipped PREFERRED CVM legitimately does not cover the far-field corners (ASAGI edge-clamps them).

5. **Plasticity derivation** — port unchanged, including the comment explaining that  $c = 1e-4 \mu$  being linear in  $\mu$  is what makes the ASAGI-interpolated cohesion exactly consistent with the element's shear modulus, and that `bulkFriction` is  $\tan(\phi)$ , the coefficient, not degrees.

### Edge Cases to Handle

- `lambda <= 0` at any source node → raise, reporting the node index,  $V_p$ ,  $V_s$ ,  $\rho$ . This is a bad CVM, not a code bug, and the message must say so.
- A slice with a different lon/lat grid than its siblings → raise; the legacy code assumes a common grid.
- A missing depth level (the CTM is missing 15200 m) → **warn**, not fail; the output node is the exact linear interpolant of its neighbours. Keep the legacy WARN.
- `extend_z_top` smaller than the mesh's maximum elevation → hard-fail via M3, naming the shortfall. This is the topography-coverage failure.

### Acceptance Criteria

- ☐ `MaterialStage.build(safs_alt, ...production params...)` writes a file **byte-identical** to the shipped `safs_material_cvm.nc`.
- ☐ `plasticity=PlasticitySpec(30, 40)` writes a file byte-identical to `safs_plasticity_phi30_40.nc`.
- ☐ The `layered_1d` source on the demo project produces an nc passing M1–M5.
- ☐ M3 correctly reports PASS for the ALT mesh and WARN (not FAIL) for the PREFERRED mesh's far-field corners against the shipped CVM — matching the measured  $x$ :  $\leq 12.2$  km,  $y$ :  $\leq 10.3$  km overhang recorded in `safs_material_cvm.yaml`.
- ☐ A synthetic CVM with one `lambda <= 0` node raises with that node's index.

- ❑ The `ctm_slices` path rebuilds a thermal nc **byte-identical** to the shipped `safs_thermal_T.nc`.
- ❑ `sv_profile` derived from the rebuilt CVM reproduces the shipped `density_profile.npz` arrays to within float64 round-off (the fixture test).
- ❑ Changing the material nc invalidates the cached Sv profile — asserted by building twice with different `grid_dx` and checking the cache key differs and the profile is recomputed.

## Dependencies

- Depends on: Phases 0, 1.
- Required by: **Phase 2** (which needs `sv_profile`), **Phase 3** (which needs the thermal nc on the `ctm_slices` path), Phase 5 gate E, and Phase 6.

This makes Phase 4 an earlier dependency than the original draft assumed: the stress and friction stages consume material-derived products, so Phase 4 must land before Phases 2 and 3 can be exercised **from raw**. They can still be developed first against the shipped ncs; only the end-to-end raw path needs the ordering.

## 1.12 Phase 5: The mesh stage – ingest and gate in code, build and improve via the skill

**In one sentence:** After this phase, the workflow can take a mesh built any way you like, convert and check it thoroughly — and meshing itself is handed to a shipped skill document that drives Claude, instead of to a library API that could never cover it.

### Goal

This is the asymmetric stage, and the asymmetry now has an answer. Mesh **generation and improvement** is an interactive, judgement-heavy craft — eighteen forked campaign folders prove it does not reduce to a function signature. But it is already written down: the `code-mesh-build-improve` skill encodes the whole pipeline, the gates, and — most valuable — the failures not to repeat. So the split is:

Concern	Who owns it	Why
<code>.msh</code> ↔ <code>.puml.h5</code> conversion	<code>deckbuild/mesh.py</code>	one exact format contract, testable byte-for-byte
Acceptance gates on a finished mesh	<code>deckbuild/mesh.py</code>	mechanical, and the deck needs them anyway
Deck-compatibility check (Stage F)	<code>deckbuild</code> (called by the skill)	it needs the stress/friction/material ncs, which only <code>deckbuild</code> builds
Building a mesh from source geometry	<b>the skill + Claude</b>	per-campaign judgement; no stable API exists
Improving quality, refining to a spec	<b>the skill + Claude</b>	iterative, non-deterministic, needs a human in the loop

The skill is therefore a **first-class deliverable of this plan**, shipped inside `integrated_workflow/`

and shared alongside the notebook. A colleague gets a notebook that builds three of the four ingredients and a skill that helps them build the fourth.

### Files to Create

- `deckbuild/mesh.py` — MeshStage: `.msh` ↔ `.puml.h5` conversion, gates A–E, and the two new ingestion gates.
- `deckbuild/stage_f.py` — the deck-compatibility check the skill’s Stage F calls.
- `skills/code-mesh-build-improve/SKILL.md` — the vendored mesh skill (see requirement 5).
- `MESHING.md` — the mesh chapter of the workflow: what the skill does, when to reach for it, and how its output re-enters the notebook.
- `tests/test_mesh.py`.

### Files to Modify

- None. `deckbuild/mesh_gen.py` — a reference generator — was in an earlier draft of this plan and is **deliberately dropped**. `run_z0cut_meshing.py` and the `gms` hybrid route are already reference implementations, and the skill points at them by path. Writing a third, weaker generator would create a maintenance burden and imply a generality it would not have.

### Detailed Requirements

1. **Port `msh_to_puml.py` and `puml_io.py` into `mesh.py`**, keeping the format contract exactly: the dataset names and dtypes, the boundary = `sum_i code_i << (8*i)` packing, the face map `f0={0,2,1}` `f1={0,1,3}` `f2={1,2,3}` `f3={0,3,2}`, the file attributes, and `orient_tets_positive`. The `tag→BC` map comes from `MeshSpec.tag_to_bc`.
2. **Fix the `int64` face-key overflow**. The legacy packed-key construction dies above  $\sim 2.1\text{M}$  nodes, and the Counter-based validator needs  $>12$  GB. Replace with a structured-view `np.searchsorted` over a `lexsorted (n0,n1,n2)` array. This is the one intentional behaviour change in the whole plan, and it must be shown to produce **identical output** on a mesh small enough for both paths.
3. **MeshStage.verify — gates A–E**, names preserved from `verify_deep40km.py`:

A	bbox, node count, tet count, fault facet count	report-only
B	fault edge max < <code>cfg.mesh_gates.fault_edge_max_m</code>	HARD
C	BC round-trip: tagged faces == geometric hull + fault*2	HARD
D	inverted tets == 0	HARD
E	volume resolution <code>f = Vs/dx &gt;= cfg.mesh_gates.f_gate_hz</code>	HARD when enabled

E needs a material `nc`, so its signature takes one; absent, E reports skip. E must also support the derived-mesh form — “introduce no NEW failures versus a frozen parent mesh” — because refinement can *create* gate failures (the `Vs` step at `z = -250 m` plus nearest-grid barycenters).
4. **Two new ingestion gates** that the legacy verifiers assume rather than check:

F	every fault facet centroid is inside the stress-nc and friction-nc hulls	
↪	HARD	
G	no pickpoint/receiver lies within <code>cfg.mesh_gates.partition_tol_m</code> of a fault facet edge shared by the mesh's coarse partition boundary	WARN

Gate G is the pickpoint rename race that aborts runs. A warn is honest: we cannot know the partition before SeisSol runs, so this is a heuristic screen, and its detail must say so.

5. **Vendor the mesh skill and make it portable.** Copy `~/claude/skills/code-mesh-build-improve/SKILL.md` into `integrated_workflow/skills/code-mesh-build-improve/`. Two edits, and only two:

- **Reference paths become relative and clearly labelled.** The skill's "Reference implementations" table and its campaign-tools list point at absolute SAFS paths (`project_7.0_preferred/meshing_deep19km/code/...`). Prefix that table with one line saying these are the *SAFS worked examples* and are read-only exemplars, not a dependency — the method transfers, the paths do not.
- **The tag contract becomes a pointer to the descriptor.** The skill hardcodes `fault=101 / top=102 / bottom=103 / sides=104 → BC 3/1/5/5`. Add a sentence: these are `MeshSpec.tag_to_bc` defaults in the project descriptor, and a different fault system may set its own.

Everything else — Stages A–F, the campaign architectures, the diagnosis playbook, the tricks that work and the verified failures — ships **verbatim**. It is the most valuable document in this plan and must not be paraphrased.

6. **Write MESHING.md** as the bridge between the notebook and the skill. It must state the canonical workflow in one page so a reader knows what they are getting into:

A	source geometry → clean fault surfaces	GOCAD/.ts/.inp → per-strand STL; ↪ CGAL
		corefine if strands butt; ↪ trims/deepening
B	surface conditioning	LEB refine to max-edge; feature-aware short-edge collapse; tetgen -d self-check ↪ quality is WON here, not in the ↪ volume
C	volume fill	tetgen -Y (rebuilds) or gmsh hybrid (from-scratch with topography)
D	volume cleanup ↪ keep best)	mmg (NON-deterministic: N draws,
		→ de-sliver → reorient negative tets
E	tags, PUML, gates	SAFS tags → msh_to_puml --validate → fault-edge / conformity / quality ↪ gates
F	deck-compatibility verification ↪ ncs	against the CONSUMING deck's actual

Plus the three load-bearing rules a newcomer will otherwise violate: **Gmsh v2.2 only; never move the fault surface; work in a separate meshing\_<variant>/ folder.**

7. **Automate Stage F, because it is the seam.** Stage F is where meshing meets this workflow: the skill tells the user to load the candidate mesh with the legacy `combined_workflow/pipeline.py` and check it against the consuming deck. Replace that

with one entry point in `deckbuild/stage_f.py`:

```
def verify_mesh_against_deck(cfg, mesh_path, deck_dir) -> GateReport
```

which checks, per the skill's Stage F list:

- 0 non-finite stress / friction / material samples at every fault facet centroid;
- `min sigma_n > 0` and `min tau_0 > 0` (else RS `psi_init = -inf`);
- `max mu_app < f0` (no pre-slip at `t=0`);
- hypocentre snap distance to the nearest facet;
- every pickpoint within tolerance of the fault; every receiver under its **local** free-surface triangle by barycentric containment — vertex proximity fails where the far-field top is 10 km coarse;
- the gate corridor `kappa` unchanged versus the frozen parent mesh.

This is the one piece of the mesh workflow that genuinely belongs in code, because it is the only piece that needs the other three ingredients.

**Share the implementation with Phase 6.** These checks overlap the deck pre-flight battery almost exactly — Stage F's `finite/sigma_n/tau_0/mu_app` checks *are* P2 and P4, and its hypocentre and receiver checks are P7 and P8. Write each check once, as a function taking `(fault, ncs)`, and have both callers compose them. The difference is only *when* they run: Stage F against a **candidate mesh** with a deck's `ncs`, before a deck exists; P1–P8 against an **assembled deck**. Two batteries, one set of predicates — otherwise they will drift and disagree, which is worse than having only one.

8. **Document the boundary explicitly.** `mesh.py`'s module docstring must state that generation, corefining, metric-driven refinement, `mmg` optimisation and fault-band grading are **not** in this module, and point at `MESHING.md` and the skill. An honest boundary is worth more than a leaky abstraction.

### Edge Cases to Handle

- A `.msh` in v4 format → raise naming the format and the `-format msh22` fix, with the misleading `vertices indices` are not unique symptom quoted so the message is searchable.
- A mesh with `>2.1M` nodes → must take the `searchsorted` path and must not allocate a `Counter`. Assert peak memory in the test via `tracemalloc`.
- A tag in the `.msh` that is not in `tag_to_bc` → raise listing the unmapped tags. Silently dropping a tag produces a mesh with a missing boundary condition.
- Gate E with no material `nc` → skip, never pass.
- `verify_mesh_against_deck` pointed at a deck whose `ncs` were built from a *different* descriptor → refuse, naming both descriptor hashes. Checking a mesh against the wrong deck's fields is worse than not checking it.

### Acceptance Criteria

- ☐ `MeshStage.to_puml` on a small SAFS `.msh` produces a file byte-identical to the one the legacy `msh_to_puml.py` produces.
- ☐ The `searchsorted` face-key path and the legacy packed-key path agree exactly on a `<2.1M`-node mesh.

- ❑ Gates A–E reproduce `verify_deep40km.py`'s verdicts on the shipped `safalt_0d5Hz_p3_deep40km.puml.h5`.
- ❑ `v4 .msh` and `unmapped-tag` inputs both raise with the specified messages.
- ❑ `verify_mesh_against_deck` reports all-PASS on the shipped `safalt_0d5Hz_p3_deep40km.puml.h5` against its own deck, and FAILs with a named cause on a mesh whose fault pokes outside the friction-nc hull.
- ❑ The vendored `SKILL.md` differs from the source only in the two edits of requirement 5 — asserted by a test that diffs the two files and allows exactly those hunks.
- ❑ `MESHING.md` names all six stages and the three load-bearing rules; checked by a test that greps for the stage letters and the rule keywords.

## Dependencies

- Depends on: Phases 0, 1. Gate E and `stage_f.py` depend on Phases 2, 3, 4 (they need the built ncs).
  - Required by: Phase 6.
- 

## 1.13 Phase 6: Deck assembly and the pre-flight battery

**In one sentence:** After this phase, the four artifacts are written into a runnable deck folder together with their YAML wiring, and a single command checks the assembled deck the way we currently check it by hand.

### Goal

Emit the deck, then run the cross-component checks that only make sense once all four pieces exist. This is where the workflow stops being four builders and becomes one workflow.

### Files to Create

- `deckbuild/deck.py` — `DeckStage`: YAML emission, `parameters.par` templating, receiver emission, and the pre-flight battery.
- `deckbuild/templates/` — Jinja-free string templates for the four wiring files:
 

<code>safs_material_cvm.yaml</code>	<code>safs_fault.yaml</code>
<code>safs_initial_stress.yaml</code>	<code>parameters.par</code>
- `tests/test_deck.py`.

### Detailed Requirements

1. **`DeckStage.assemble(cfg, deck_dir, artifacts, spec) -> Path`** writes a deck folder: copies the ncs and the mesh, renders the wiring files, writes the receiver and pickpoint lists, and writes `deck_manifest.json`.

**Copy, never symlink.** A deck is a shippable unit that gets rsynced to Frontera or Expanse; a symlink into `outputs/` either breaks on transfer or silently ships whatever that path later points at. Copy and verify by sha256 after copying.

- 1b. **The filename contract — every generated file has one declared destination.** This is the mapping `assemble` implements, and it belongs in the README verbatim. Build dirs are relative to `outputs/<project>/<run_tag>/`:

artifact	built in	lands in the deck as
material nc	material/	<prefix>_material_cvm.nc
plasticity nc	material/	<prefix>_plasticity_<phi>.nc
thermal nc	thermal/	(NOT shipped -- a design input only)
stress nc	stress/	<prefix>_stress_<design>.nc
friction nc	friction/	<prefix>_friction_<case>.nc
rs_muw LuaMap	friction/	(inlined into <prefix>_fault.yaml)
mesh	(from the skill)	<mesh_name>.puml.h5
receivers	./	<prefix>_receivers.dat
pickpoints	./	<prefix>_pickpoints.dat

which deck file references it

```

material nc, plasticity nc -> <prefix>_material_cvm.yaml
stress nc -> <prefix>_initial_stress.yaml
friction nc, rs_muw -> <prefix>_fault.yaml
mesh, receivers, pickpoints -> parameters.par

```

Three rules the implementation must enforce:

- **<prefix> comes from the descriptor**, not hardcoded `safs_`. A colleague's deck is not named after the San Andreas.
- **The name encodes the design** (k1.7, case1, phi30\_40), because deck folders get compared against each other and an ambiguous filename has caused a real mix-up.
- **The YAML file: field and the copied filename are written from the same variable**. They cannot be allowed to drift; P1 then re-checks the result on disk.

1c. **DeckStage.resolve(artifacts)** -> **dict** returns the full path map without writing anything, so the notebook can print exactly where every file will go *before* assembling. This is what makes “where did my file end up” answerable at a glance.

2. **Templates preserve the deck's most valuable feature: its comments**. The shipped YAMLs carry long provenance headers explaining *why* each choice was made. The templates must emit a generated-provenance header — descriptor name, descriptor sha256, every build parameter, the timestamp — and then leave a clearly marked block for hand-written notes that is preserved across regeneration.

3. **Pre-flight battery P1–P8**, all reading the **assembled deck's live file: targets**, never in-memory arrays:

P1 every `file:` referenced by every yaml exists NEXT TO the yaml, is a real file (not a symlink), and its sha256 matches deck\_manifest.json. Also: every file IN the deck folder is accounted for in the manifest -- an unexplained extra nc is how a stale field gets shipped. HARD

P2 on-fault projection:  $\sigma_n > 0$  and  $\tau_0 > 0$  on every facet HARD  
(this is the `psi_ini = -inf` abort)

- P3 `psi_ini` finite everywhere:  
`psi = a*ln[(2*sr0/V_ini)*sinh(tau_0/(a*sigma_n))];` and `mu_ini ==`  
`tau_0/sigma_n` HARD
- P4 no facet is pre-slipping at `t=0`: `max(mu_app) < min(mu_s or f0)` HARD
- P5 plasticity sub-yield at `t=0`: 0 grid points yield; report `max tau/taulim`  
and the binding point HARD if  
`plasticity on`
- P6 gate corridor: `kappa_bar(s)` with the coasting window; longest  
sub-`kappa_c` run < `gap_max_km`; per named band WARN
- P7 nucleation: the `Tnuc_s` Lua centre equals the SNAPPED hypocentre; snap  
distance reported; `S_E = a*sigma_n*ln(V_dyn/V_init)` vs the overstress  
amplitude; `L_b`, `L_nuc`, and mesh resolution of `L_b` ( $\geq 4$  elements) HARD
- P8 output hygiene: no receiver at `z = 0` exactly; `OutputRegionBounds`  
contains no literal `0.0`; `wavefieldoutput = 1` HARD
- P6 is a WARN, deliberately.** `kappa` is necessary, not sufficient — an `f_w = 0.10` design passed this screen and still arrested at `s ≈ 60 km`. The detail string must say so, so nobody reads a green P6 as a prediction.
  - Receiver emission.** Port `pgv_receiver_map`'s station-table writer and `onfault_receivers`' pickpoint writer. Enforce `z = -1 m` for surface receivers (P8), and provide `drop_ids` so a pickpoint that triggered the rename race can be removed by id.
  - `DeckStage.diff(deck_a, deck_b) -> report`** — list which of the deck's files differ, by sha256, so an A/B pair can be shown to differ in exactly one input. Several past experiments were confounded by an unintended second change; this makes that visible.

### Edge Cases to Handle

- Two artifacts built from different descriptors → refuse to assemble, naming both descriptor hashes. This is the “stale outputs/” failure mode.
- A deck with plasticity fields but `Plasticity = 0` in `parameters.par`, or the reverse → hard fail. Both directions are silent wrong-physics.
- Regenerating over an existing deck → refuse unless `overwrite=True`, and never overwrite the hand-written notes block.
- P7 where the hypocentre lies outside the fault surface → hard fail. This happened once (the ALT nucleation was off-fault and had to be re-snapped).

### Acceptance Criteria

- ☐ assemble on the four SAFS ALT artifacts produces a deck whose four ncs are byte-identical to a shipped deck's, and whose YAML file: targets and active (non-comment) YAML content match the shipped YAMLs.
- ☐ P1–P8 all report PASS on that shipped deck.
- ☐ P2 reports FAIL on a deliberately unpatched daylighting stress nc, and its detail names the failing facet count.
- ☐ P5 reproduces the recorded `max tau/taulim = 0.6355` at `(399000, 3750000, -1500)` for the phi30/40 deck.
- ☐ P8 catches a planted `z = 0` receiver and a planted literal `0.0` in `OutputRegionBounds`.

- diff on two decks differing in one nc reports exactly one changed file.

## Dependencies

- Depends on: Phases 2, 3, 4, 5.
  - Required by: Phase 7.
- 

## 1.14 Phase 7: The integrated notebook and the demo project

**In one sentence:** After this phase, a colleague can open one notebook, press Run All, and watch a complete deck get built and checked for a small example fault — then swap in their own.

### Goal

Write the deliverable. The notebook is thin by design: all the logic is in deckbuild, so the notebook is narrative plus parameter cells.

### Files to Create

- deck\_workflow.ipynb — the integrated notebook.
- projects/demo\_planar.yaml — the shipped example, filled in.
- deckbuild/demo.py — synthetic *field* generators for the demo (layered 1-D velocity, constant orientation, depth-profile friction). No mesh generation.
- data/demo/demo\_planar.puml.h5 — the pre-built demo mesh, committed (< 20 MB).
- download\_data.py — fetch the SAFS data/ inputs; the demo needs none of them.
- README.md — filled in.

### Detailed Requirements

1. **Notebook structure** — eight sections, each ending in a printed GateReport:

[0]	Setup and project	bootstrap.init(project=...); print the descriptor
	↪ AND an	inventory of the raw data found, with row counts
[1]	What we are building	the data-flow diagram; print the resolved output
	↪ paths for	EVERY artifact this run will write, before writing any
[2]	Mesh	INGEST a mesh → convert / verify / plot (gates A-E, F, G) + snap the hypocentre onto it and plot where it landed + a pointer to the mesh skill for building or improving one
[3]	Material	RAW CVM → nc; + plasticity; + Q; + thermal nc from RAW CTM; + the derived Sv(z) profile (gates ↪ M1-M5)
[4]	Stress	RAW CSM orientation → grid → Sv → closure → nc (gates V1-V4)
[5]	Friction	thermal nc or depth profile → a, V_w; graded f_w → Lua (gates F0-F6, FL, G1-G4)
[6]	Assemble	deck folder + yamls + receivers (gates ↪ P1-P8)
		+ print the final path map and diff it against section [1]'s ↪ prediction

[7] Design read-outs Mw estimate, kappa corridor, nucleation, yield margin

Section [3] must come **before** [4] and [5] and the notebook must say why: the stress closure needs  $S_v(z)$  from the material, and the friction zoning needs the thermal nc. Running them out of order is the most likely way a user pairs a new CVM with a stale stress field.

2. **Every section has exactly one PARAMETERS cell**, at its top, containing only assignments with inline comments. No logic. This is the cell a user edits. The hypocentre lives in section [2]’s cell and is written the way a user actually has it:

```
HYPOCENTER = dict(lon=-116.35, lat=33.78, depth_m=10000.0) # or x=, y=, z=
HYPO_SNAP_TOL_M = 2000.0 # hard-fail if the fault is further away than this
```

The notebook then prints the snap distance, the resulting  $s_{\text{km}}$  and depth, and **plots the requested and snapped points on the fault** so the user can see it landed where they meant. Everything downstream — the  $T_{\text{nuc}}$  Lua, F1, P7, the Mw estimate — uses the snapped point.

3. **Every section’s markdown states the physics before the code**: the equation being evaluated, the sign convention, and the one gotcha most likely to bite. Pull these from the exploration doc §9 — do not re-derive them.
4. **Section [2] is honest about what it does**. It does **not** generate a mesh. It ingests one, converts it, gates it, and — once the other stages have run — calls `verify_mesh_against_deck`. Its markdown must say, in one short paragraph: *to build a mesh for your own fault, or to improve this one, open the code-mesh-build-improve skill in skills/ with Claude; MESHING.md is the one-page map*. Do not bury this in a footnote — for a colleague on a new fault system it is the most important sentence in the notebook.
5. **The demo project must run end-to-end with zero downloads and no gmsh**. A planar 60 km  $\times$  15 km vertical strike-slip fault, a three-layer 1-D velocity model, uniform SHmax, a depth-profile friction law, and a **small pre-built mesh shipped in data/demo/** (a coarse planar-fault .puml.h5, target < 20 MB, committed). Shipping the mesh rather than generating it keeps gmsh off the critical path, makes the demo deterministic, and is consistent with section [2] ingesting rather than generating. Target: full Run-All under 5 minutes on a laptop. If it cannot hit that, shrink the fault, not the number of stages.
6. **The demo mesh is itself a worked example of the skill**. Record how it was made — the stages used, the gate numbers it achieved — in `MESHING.md`, so a reader sees one complete small instance of the pipeline before attempting their own.
7. **A “retarget this to your fault” section** at the end of the README, as a checklist keyed to descriptor fields. Split it three ways, because the three kinds of work are not alike:

You must supply	You may keep	You must decide yourself
fault surface geometry; velocity model; stress orientation (or a constant); friction parameterisation; hypocentre; strike frame and CRS <b>a mesh</b> — built with the skill, not by this notebook	every physics default in <code>PhysicsDefaults</code> the gate thresholds	the tectonic regime — the Andersonian closure assumes <code>sigma2</code> vertical, i.e. strike-slip how much mesh resolution you can afford

8. **Keep the existing deep-dive notebooks.** `combined_stress_friction_check.ipynb`, `combined_graded_fw_check.ipynb`, `initial_state_variable.ipynb`, `plasticity_roten2014.ipynb` stay where they are and are linked from the integrated notebook as “go deeper on this stage”. The integrated notebook is the map, not a replacement.

### Edge Cases to Handle

- A user opens the notebook with `project="safs_alt"` but no data/ → `require_files=True` raises at the bootstrap cell with the `download_data.py` hint. It must not fail three sections later.
- Colab: the pip cell must run before any `deckbuild` import, and the Drive search must report which candidate folder it chose.
- A stage whose gates FAIL must still let the notebook continue to the next section, printing a loud banner. Only assemble refuses to proceed on a hard failure.

### Acceptance Criteria

- ☐ `jupyter nbconvert --execute deck_workflow.ipynb` with `project="demo_planar"` completes with a non-zero exit only if a gate fails; all demo gates pass; wall time < 5 min.
- ☐ The same with `project="safs_alt"` reproduces the shipped deck (this re-runs Phase 6’s criterion through the notebook).
- ☐ Every section prints a `GateReport`; asserted by a test that parses the executed notebook’s outputs.
- ☐ The demo runs **entirely from raw**: no pre-built nc is read, asserted by a test that renames every `*.nc` under `data/demo/` before executing and expects success.
- ☐ Section [1]’s predicted path map equals section [6]’s actual path map, asserted by parsing both printed tables from the executed notebook.
- ☐ Changing `HYPOCENTER` and re-running changes the `Tnuc_s` Lua centre, the reported `s_km`, and `P7`’s numbers — asserted by executing the notebook twice with different hypocentres and diffing the emitted fault YAML.
- ☐ `README.md` retarget checklist names every required descriptor field, cross-checked against `Project`’s `dataclass` fields by a test.

### Dependencies

- Depends on: Phase 6.
- Required by: Phase 8 (the SAFS reproduction exercise).

## 1.15 Phase 8: The SAFS reproduction exercise – rebuild a production deck from raw

**In one sentence:** After this phase we have *proof*, not a claim: the workflow, fed the same raw CVM, CTM and CSM data we started from, rebuilds a deck we are running today, and every field matches value-for-value.

### Goal

This is the exercise that decides whether the workflow is trustworthy. Everything before it is machinery; this is the demonstration. It also produces the single most useful artifact for a colleague adopting the workflow: **one complete, real, verified worked example**, from raw data files to a runnable deck, with every number checked against a deck that has already run on Frontera.

Do it on SAFS, the system we know, *before* attempting a second fault system in Phase 9. If the workflow cannot reproduce the deck it was extracted from, nothing else matters.

### The target

```
deck: safs_seisol_v4_0_0_RSSRW_ALT_THERMAL_CASE1_intermediate
 _plast_phi30_40_gradedfw_k1p70_nwredM7p8_sefw0_attenuation_deep40km
```

Chosen because it exercises **every** feature the workflow has: a rebuilt-from-raw CVM, plasticity, attenuation, CASE1 thermal friction, a constant  $k = 1.70$  stress field, a 7-region graded  $f_w$  !LuaMap, a compact-bell nucleation, PGV receivers, and a depth-extended mesh. A deck that exercised fewer paths would prove less.

Its inventory, and where each piece comes from:

piece		built from	exercise step
-----		-----	-----
safalt_0d5Hz_p3_deep40km.puml.h5		INGESTED (not rebuilt)	E0
safs_material_cvm.nc	362 MB	37 raw CVM slices	E3
safs_plasticity_phi30_40.nc	242 MB	the material nc, phi 30/40	E3
(thermal, not shipped in deck)		105 raw CTM slices	E3
safs_stress_andersonian_k1.7.nc	89 MB	CSM orientation + $S_v$ + $k=1.70$	E4
safs_friction_thermal_case1.nc	42 MB	thermal nc, CASE1 zoning	E5
safs_fault_rs_muw...yaml		7-region graded $f_w$ design	E5
safs_*.yaml, parameters.par		templates + the descriptor	E6
safs_pgv_receivers_100k.dat		receiver design	E6
safs_pickpoints.dat		pickpoint design	E6

**Raw data availability is confirmed** (checked 2026-08-01, all present in this tree):

```
raw_data/multiscale_statewise_cvm/CVM*_h_data.csv 37 slices (muscal)
thermal/raw/CTM*_h_data_final.csv 105 slices (shinevar2024)
raw_data/yang_and_hauksson_orientation/CSM_data_1782324800027.csv (YHSM-2013)
```

One useful fact already established: that CSM csv is **md5-identical** to the CSM\_orientation.csv shipped in combined\_workflow/data/, so the orientation path has no hidden preprocessing step — it is a straight copy. Confirm the analogous question for the CVM and CTM slices as step E1; do not assume it.

## Files to Create

- `deckbuild/introspect.py` — read a shipped deck, emit a draft descriptor and build parameters (see requirement 2).
- `projects/safs_alt_v4_repro.yaml` — the descriptor for this exercise.
- `exercise_safs_reproduction.ipynb` — the exercise, runnable top to bottom.
- `EXERCISE_safs_reproduction.md` — the results document (see requirement 6).
- `tests/test_repro_safs.py` — the ladder below, as tests, skipped when the decks are absent.

## Detailed Requirements

1. **Run the exercise as a ladder, E0–E6, and stop at the first rung that fails.** Each rung is independently meaningful, so a partial result is still evidence. Do not attempt E6 before E1–E5 pass — a full-chain mismatch is nearly impossible to localise.

- E0 INGEST the deck's mesh; gates A–E pass; facet count matches what the legacy pipeline reports for the same file.
- E1 PROVENANCE: do the raw slices here match the shipped ncs? Check slice count, depth levels, lon/lat extent and column names against the nc's axes and attributes. A mismatch means the raw data was re-exported since -- report that, do not chase a difference the exercise cannot close.
- E2 DESCRIPTOR: introspect the deck → draft descriptor; diff it against `projects/safs_alt.yaml`; reconcile every difference.
- E3 MATERIAL: raw CVM → material nc; + plasticity  $\phi$  30/40; raw CTM → thermal nc; +  $S_v(z)$  vs the legacy profile  $npz$ .
- E4 STRESS: CSM orientation + E3's  $S_v$  +  $k=1.70$  → stress nc.
- E5 FRICTION: E3's thermal nc + CASE1 → friction nc; + the 7-region `rs_muw` LuaMap.
- E6 DECK: assemble; P1–P8; then diff the assembled deck against the shipped one, file by file.

2. **`deckbuild.introspect` — recover the descriptor from a deck.** This is what makes E2 tractable, and it is independently useful to anyone adopting the workflow with decks they already have:

```
def introspect_deck(deck_dir) -> (Project, dict)
returns a DRAFT descriptor and the build parameters it inferred
```

It reads: each nc's axes (to recover the grid boxes and spacings), the easi YAMLS' file: targets and !ConstantMap values (to recover `rs_b`, `rs_sl0`, the far-field fallback), the `rs_muw` Lua (to recover the strike frame, the region values and the transition windows — they are literals in the emitted code), the `Tnuc_s` Lua (to recover the hypocentre, radius and amplitude), and `parameters.par` (to recover Plasticity,  $T_v$ , the attenuation band, and the output settings).

It must **mark every field it could not infer** rather than guessing — the output is a draft a human completes, and a silently-wrong default here would poison the whole exercise.

3. **Compare with `compare_nc`, and report both tiers.** For each of E3, E4, E5 record `data_identical` (the criterion) and `file_identical` (informational). On a data mismatch,

report the variable, the flat index, both values, and the max absolute and relative difference over the whole field — a one-ULP difference in the last z-slice and a wholesale sign flip must not look the same in the report.

4. **Declare up front what cannot match, so a difference there is not mistaken for a bug.** Put this table in the exercise notebook *before* the first comparison:

WILL match, value-for-value

every nc variable and axis; the rs\_muw Lua's numbers; the active (non-comment) content of every easi yaml; the receiver and pickpoint coordinates

WILL NOT match, by design

the mesh	-- INGESTED, never rebuilt (Phase 5 scope)
yaml comments	-- the shipped headers are hand-written provenance prose
parameters.par	-- machine/campaign-specific (nodes, paths, EndTime)
nc header bytes	-- netCDF4/HDF5 library version, not data

MIGHT NOT match, and each is a real finding to run down

material nc	-- scipy's Delaunay/Qhull can differ across versions, which would perturb LinearNDInterpolator at the last ULP
Sv(z) profile	-- depends on the material nc matching first
anything downstream	-- a material diff propagates into stress+friction

The third category is the honest one. If the material nc differs at the last ULP because of a scipy version, say so, quantify it (max relative difference over the grid), and then **re-run E4/E5 against the shipped material nc** to prove the stress and friction stages are themselves exact. Do not let an upstream environment difference be reported as a failure of the downstream stages.

5. **Recover the design parameters, do not assume them.** The exercise must *read* these off the shipped deck and state them, because at least one is not what a reader would guess:
  - $k = 1.70$ , **constant** — the magnitude lever in this deck is the graded  $f_w$ , not a graded  $k$ .
  - `freeze_above_depth_m` — the shipped stress nc is named `safs_stress_andersonian_k1.7.nc` with `no_freeze{D}m` tag, which by the Phase 2 naming rule means the freeze is **off** for this deck. Verify that against the field itself (compare the shallowest slices to the freeze-depth slice) rather than trusting the filename.
  - the 7  $f_w$  values and 6 transition windows, from the rs\_muw yaml's own filename and its Lua body — they must agree.
  - $\phi_{\text{soft}} = 30$ ,  $\phi_{\text{hard}} = 40$  — *not* the Roten paper's 35/45, which is the PlasticitySpec default. This is exactly the case the Phase 4 “neither may be the silent default” rule exists for.
  - the hypocentre (604446.944, 3704576.3853, -10067.9819),  $R = 2000$  m, 75 MPa.
6. **EXERCISE\_safs\_reproduction.md is the deliverable.** A results document with: the ladder table (rung, verdict, evidence), the recovered parameter list, the two-tier comparison numbers per field, every difference found with its explanation and category, and the wall time per stage.

This doubles as the worked example shipped with the workflow — so write it for a reader who has never seen SAfs.

7. **Feed the findings back.** Anything this exercise discovers that the plan got wrong — a missing descriptor field, a parameter that could not be recovered, a check that should have caught a difference earlier — is fixed in the relevant phase, not patched around here. The exercise is a test of the design, and a design bug found here is the cheapest one we will ever find.

### Edge Cases to Handle

- The raw CVM slices in this tree turn out **not** to be the ones the shipped nc was built from (E1 fails) → stop, report it, and run E4–E6 from the *shipped* material nc instead. The exercise then proves everything except the CVM rebuild, which is still a real result.
- A rung fails for an environment reason (scipy version) → quantify, categorise, continue down the ladder using the shipped upstream artifact.
- The introspected descriptor disagrees with the hand-written one → the hand-written one is not automatically right. Reconcile against the *deck*, which is ground truth.
- The exercise needs ~1.7 GB of deck files plus the raw data → gate the tests behind the `--decks=PATH` option like the rest of the byte-exact suite, and make the notebook fail at its first cell with a clear message when they are absent.

### Acceptance Criteria

- ☐ E0–E6 all reported, each with a verdict and evidence, in `EXERCISE_safs_reproduction.md`.
- ☐ E3, E4, E5 report `data_identical = True` against the shipped ncs — or, where they do not, a quantified explanation in the “might not match” category plus a passing re-run from the shipped upstream artifact.
- ☐ The emitted `rs_muw` Lua is `data_identical` to the shipped yaml’s Lua body.
- ☐ E6’s assembled deck differs from the shipped deck **only** in the declared “will not match” set — asserted by a file-by-file diff whose exceptions are an explicit allowlist, not a filter.
- ☐ `introspect_deck` on the target deck recovers `k`, the `f_w` design, `phi_soft/hard`, the hypocentre and the attenuation band, and marks as unknown anything it cannot infer.
- ☐ `exercise_safs_reproduction.ipynb` runs top to bottom via `nbconvert --execute` with the decks present.
- ☐ Every discrepancy found is either fixed in its owning phase or recorded in the results document with a reason it is acceptable.

### Dependencies

- Depends on: Phases 0–7 (it uses all of them).
- Required by: Phase 9. Do not start the second-system work until the first system reproduces — a generality claim on top of an unverified base is worthless.

---

## 1.16 Phase 9: Second-system validation and legacy retirement

**In one sentence:** After this phase we have evidence the workflow really is fault-system-agnostic, and the old code is either retired or explicitly kept.

## Goal

Prove generality with a real second case, then decide the fate of the legacy tree.

## Detailed Requirements

1. **Build a deck for the second fault system that is already in this tree.** `seisolv_quakeworx/tpv13/` is a complete, working SCEC TPV12/13 deck — a 60° dipping **normal** fault — and it ships everything Phase 9 needs: `tpv13_training.puml.h5`, `tpv13_training.msh`, `tpv13_training.geo`, `tpv12_13_{material,initial_stress,fault}.yaml`, `parameters.par`, and `tpv13_qwx.ipynb`. No new data has to be found. Deliverable: `projects/tpv13.yaml` plus a notebook section that rebuilds this deck through `deckbuild` and diffs it against the shipped one.
2. **TPV13 will not fit the Andersonian closure, and that is the point.** Its stress is an analytic depth-dependent `!LuaMap`, not an ASAGI grid:

```
s_max_minus_Pf = 9.8 * (2700 - 1000) -- effective overburden grad
depth <= 11951.15 m : s_zz = -s_max_minus_Pf * depth
 s_xx = -round2(0.5*(1+0.3496)*s_max_minus_Pf) * depth
 s_yy = -round2(0.3496 *s_max_minus_Pf) * depth
deeper : s_xx = s_yy = s_zz = -s_max_minus_Pf * depth (isotropic)
```

Its material and plasticity are `!ConstantMap` (`rho 2700`, `mu 2.9403e10`, `lambda 2.941e10`, `plastCo 1e6`, `bulkFriction 0.60`). So TPV13 exercises three descriptor paths that SAFS never touches, and is therefore the right generality test:

- a **"constant" material** kind (no CVM at all);
- an **"analytic" stress** kind — a user callable returning the six components, emitted as a `!LuaMap` rather than baked to an `nc`;
- a **non-strike-slip regime**, which is exactly the assumption the Summary flags.

Add `stress.kind` to the descriptor with `"andersonian_c1"` and `"analytic"` as the two options, and implement `"analytic"`. Do **not** bend the Andersonian recipe to fit; the two recipes coexist.

3. **A `!LuaMap` output path is therefore in scope for Phase 9.** `DeckStage` must be able to emit a stress block as Lua as well as as an ASAGI reference, reusing the Lua emitter Phase 3 already builds for `rs_muw`. Acceptance: the emitted TPV13 stress Lua evaluates to the same six components as the shipped `tpv12_13_initial_stress.yaml` at 1000 fixed-seed points, including the `math.floor(x + 0.5)` rounding, which is load-bearing.
4. **Legacy tree decision.** For each of `combined_workflow/` and `stress_build_workflow/`, pick one and record it in the README:
  - *retire* — delete after the byte-exact tests pass, leaving a `MOVED.md` pointer;
  - *keep as reference* — freeze, mark read-only in the README, and state that `integrated_workflow/` is the maintained path.

Recommendation: **keep `combined_workflow/` frozen** (its post-processing notebooks are out of scope here and still used), and **retire `stress_build_workflow/`** once its eight step-notebooks are represented as the stress section's deep-dive, since its `functions/` are already a stale copy of `combined_workflow/lib/`.

5. Write `CODEBASE_GUIDE.md` in `integrated_workflow/` by merging the exploration document with the final API.

### Acceptance Criteria

- ☐ `projects/tpv13.yaml` builds a deck whose active YAML content matches the shipped `seisol_quakeworx/tpv13/` deck, and whose emitted stress Lua agrees with `tpv12_13_initial_stress.yaml` at 1000 fixed-seed points including the rounding.
- ☐ The "constant" material kind and the "analytic" stress kind are both exercised by that build.
- ☐ The full byte-exact suite passes against **at least three** shipped decks spanning ALT/PREFERRED, CASE1/CASE2 and graded-k/graded-f\_w.
- ☐ The legacy decision is recorded, and no file in `integrated_workflow/` imports from `v3_under_construction/`.

### Dependencies

- Depends on: Phase 8.
- Required by: nothing.

---

## 1.17 Testing Strategy

**The spine is data-exactness.** Every physics phase (2–5) has, as its first acceptance criterion, that it reproduces a shipped artifact byte-for-byte. This is what makes a large port safe: the test is unambiguous, needs no tolerance, and cannot be satisfied by approximately-right code.

Level	What	When
Unit	Each ported pure function against its legacy import, 0 ULP on fixed-seed inputs	Phases 1–5
Golden file	<code>assert_nc_identical</code> against a shipped deck's nc	Phases 2, 3, 4
Golden text	Emitted Lua and YAML against a shipped deck's	Phases 3, 6
Battery parity	New GateReport pass/fail vector equals the legacy verifier's	Phases 2, 3, 5
Negative	Every hard-fail path has a test that plants the defect and asserts the raise	All phases
Integration	<code>nbconvert --execute</code> on both the demo and SAFS projects	Phase 7
Reproduction	The SAFS deck rebuilt from raw and compared field-by-field	Phase 8
Generality	A second fault system end to end	Phase 9

**Test data.** Byte-exact tests need shipped decks, which are large and untracked. Gate them behind a `--decks=PATH` pytest option and `skipif` when absent, so the suite runs everywhere and is exhaustive where the data exists. The demo-project tests must never skip.

**What is deliberately not tested.** Two things. First, whether a design actually crosses a gate in a real simulation — that is decided by SeisSol, not by us; see the P6 warning. Second, the mesh skill's

*advice*. A document that teaches judgement cannot be unit-tested. What is tested is that it ships intact (the vendoring diff test) and that everything it hands off to code — conversion, gates A–G, Stage F — is verified against a shipped mesh.

## 1.18 Risk Assessment

Risk	Severity	Mitigation	Related constraint
A port silently perturbs a float, and a deck built next month differs from one built today in a way nobody notices.	<b>High</b>	Data-exact golden tests are the first acceptance criterion of every physics phase. Function bodies are copied, never retyped. Phase 8 then re-proves the whole chain from raw.	Data-exact reproduction
The SAFS reproduction exercise fails on the material nc because scipy's Delaunay changed between versions, and the whole exercise reads as a failure.	Medium	The exercise declares this category <b>before</b> comparing, quantifies the difference, and re-runs the downstream rungs from the shipped material nc — so the stress and friction stages are still proven exact. Pin scipy in <code>environment.yml</code> and record the version in every artifact's provenance.	Phase 8 requirement 4
The exercise is deferred as a chore, and the workflow ships unverified.	Medium	Named in the Summary as how we will know it works; Phase 9 is explicitly blocked on it; the closing section says to cut Phase 9 rather than Phase 8.	Phase 8; Suggested order
The mesh stage over-promises: someone assumes the notebook can produce a production mesh and discovers mid-campaign that it cannot.	<b>High</b>	Section [2] says in prose that it ingests rather than generates; <code>mesh.py</code> 's <code>docstring</code> states the boundary; the demo ships a pre-built mesh so nothing implies generation. The skill and <code>MESHING.md</code> are the named path.	Mesh split, stated in the Summary
The mesh skill goes stale: the workflow ships a vendored copy that drifts from the maintained one, or its SAFS reference paths mislead a new user.	Medium	Only two edits are allowed when vendoring, enforced by a diff test. The reference table is explicitly relabelled as read-only SAFS exemplars. Re-vendor when the source skill changes.	Phase 5 requirement 5

Risk	Severity	Mitigation	Related constraint
A colleague without a capable agent gets only half a workflow, since the mesh half assumes Claude on the other end.	Medium	Named in the Summary as the main tradeoff. MESHING.md states the pipeline as a human-followable procedure, and the eighteen campaign folders remain worked examples, so the skill is an accelerator rather than a hard dependency.	Summary tradeoff
The descriptor becomes a second place to be wrong: a field is added to the schema but the code keeps reading its old constant.	Medium	a test_no_hardcoded_constants test greps the package for the SAFS literals and fails on any hit. Field-by-field equality test against the legacy _common.py.	Phase 0 acceptance
The Andersonian closure does not generalise to a non-strike-slip regime, so “works for other fault systems” is overstated.	Medium	Named in the Summary’s <i>What this does NOT do</i> . Already largely retired as a risk: the TPV13 deck in this tree is a 60° dipping normal fault with analytic stress and constant material, so Phase 9 has a concrete second system in hand and adds an “analytic” stress kind for it.	Phase 9 requirements 2 and 3
Scope creep: the workflow grows a post-processing half and never ships.	Medium	Post-processing is out of scope; the existing notebooks stay where they are and are linked, not absorbed.	Summary scope guard
The int64 face-key fix in Phase 5 is the one intentional behaviour change and could differ from the legacy path.	Medium	Both paths are run on a mesh small enough for the legacy one and asserted identical, with a tracemalloc bound on the new path.	Phase 5 requirement 2
Rebuilding from raw is slow enough that users quietly go back to hand-copying a pre-built nc, and the raw path rots.	Medium	The expensive steps are the Delaunay interpolations; cache the reprojected slice stack under data/<project>/cache/ keyed by the raw inputs’ hashes, so only the first build pays. Measure and report the build time per stage in the notebook.	The data-flow section
A rebuilt CVM is paired with a stale stress field, because Sv did not propagate.	Medium	sv_profile is a required argument to StressStage.build, cached by the material nc’s sha256, and recorded in the stress artifact’s provenance. Notebook section order enforces material before stress.	Phase 2 requirement 3; Phase 4 requirement 3b

Risk	Severity	Mitigation	Related constraint
The hypocentre silently lands on the wrong strand or the wrong friction region.	Medium	snap_hypocenter hard-fails past a tolerance, reports the snap distance and resulting s_km, flags a band crossing, and the notebook plots requested vs snapped on the fault. Automatic strand detection is not possible — this is why the plot exists.	Phase 1 requirement 1b
A deck ships a file from a previous run, or a YAML file: points at something that is not there.	Medium	One declared destination per artifact; per-run output folders keyed by parameters; copy-not-symlink; P1 re-checks existence <i>and</i> sha256 against the manifest <i>and</i> flags unaccounted extra files in the deck.	Phase 6 requirements 1, 1b
The demo project is too slow or too large, so nobody runs it and the workflow is judged by SAFS alone.	Low	Hard budget: full Run-All under 5 minutes with zero downloads. Shrink the fault, never the stage count.	Phase 7 requirement 5
Regenerating a deck destroys the hand-written provenance comments that make the shipped YAMLS valuable.	Low	Templates emit a generated header plus a preserved hand-notes block; assemble refuses to overwrite without <code>overwrite=True</code> .	Phase 6 requirements 2 and 3

## 1.19 Suggested order and rough shape

Phases 0 and 1 are prerequisites for everything and are small. Phases 2, 3 and 4 are independent of each other and can proceed in any order or in parallel — start with 2, since it is the most mature and will shake out the contract.

Phase 5 changed shape once the mesh skill was brought in, and is now **smaller than the others, not larger**: conversion, gates, and one deck-compatibility function, plus vendoring a document. Its Stage F piece depends on Phases 2–4, so schedule it after them even though its conversion half could start earlier.

Phases 6 and 7 are the payoff and must not be deferred — a plan that lands 0–5 and stops has produced a library, not the workflow that was asked for. Phase 7 is also where the two halves of the deliverable are finally joined: the notebook that builds three ingredients, and the skill that builds the fourth.

**Phase 8 is the one to protect.** It is tempting to treat “rebuild the SAFS deck from raw and diff it” as a validation chore to be squeezed at the end. It is the opposite: it is the only step that turns a plausible-looking library into something anyone should trust, and it is where design bugs surface at their cheapest. Budget real time for it, expect it to send work back into earlier phases, and do not start Phase 9 until it passes. If schedule pressure forces a cut, cut Phase 9 — a workflow proven on

one system is useful; a workflow claimed to generalise but proven on none is not.